Metalplafer 360

MANUAL DE PUESTA EN MARCHA

Cómo montar METALPLAFER360 desde cero, paso a paso

Escrito para quien no es programador. Se explica qué hacer, dónde hacerlo, qué botón pulsar y qué copiar. No hace falta instalar nada en el ordenador: casi todo se hace desde el navegador.

Aplicación interna de gestión para Metalplafer · Sabadell
Fichas, proyectos, órdenes de trabajo, material, horas, documentos y copias de seguridad.

Qué vas a encontrar aquí

Los apartados 1 al 9 son los imprescindibles: con ellos la aplicación ya funciona. Del 10 en adelante son mejoras que puedes dejar para otro día.

1 Crear la cuenta de Supabase
Dónde vivirán los datos de la empresa

2 Crear el proyecto
Contraseña de la base de datos y región

3 Ejecutar el SQL
Los archivos que crean la aplicación por dentro

4 Las tablas
Qué se guarda y dónde, explicado en castellano

5 Los permisos (RLS)
Por qué un trabajador no puede ver lo de administración

6 El acceso de las personas
Usuarios y contraseñas, sin correos electrónicos

7 El almacén de archivos
Fotos, planos, vídeos y firmas

8 Crear el primer administrador
La primera persona que entra manda

9 Las claves de conexión
Qué se copia a GitHub y qué no se copia jamás

10 Google Drive
Para que las copias se guarden fuera de Supabase

11 Las copias de seguridad
A mano y automáticas, todos los días a las 02:00

12 Restaurar una copia
Cuando algo se ha perdido o estropeado

13 GitHub
Dónde vive el programa y cómo se publica

14 Publicar la aplicación
La dirección en la que entrará la gente

15 Entrar por primera vez
Y qué hacer los primeros diez minutos

16 Dar de alta trabajadores
Altas, contraseñas y bajas

17 Los datos de ejemplo
Para aprender sin miedo, y cómo limpiarlos

18 Actualizaciones
Qué hacer cuando llegue una versión nueva

19 Si algo no funciona
Los problemas más habituales, con su solución

Antes de empezar, ten a mano

20 Una cuenta de correo electrónico.

21 La cuenta de GitHub donde está el programa.

22 Aproximadamente **40 minutos** para los apartados 1 al 9.

La regla de oro sobre las contraseñas

En este manual vas a manejar algunas claves. Hay una que **no se pega nunca** en GitHub ni en ningún archivo del programa: la que Supabase llama `service_role`. Lo mismo vale para el archivo de credenciales de Google. Esas dos cosas solo se pegan dentro de Supabase, que es un sitio privado. Si dudas: si parece una contraseña o una clave larguísima, no va a GitHub.

1 Crear la cuenta de Supabase

Supabase es el servicio donde vivirán los datos de la empresa: los clientes, los presupuestos, las horas, las fotografías. Es gratis para un taller de este tamaño.

1. Entra en **supabase.com** y pulsa **Start your project**.
2. Regístrate con la cuenta de Google de la empresa o con un correo y una contraseña.
3. Si te pide confirmar el correo, abre el mensaje que te envían y pulsa el enlace.

> Apunta en un sitio seguro con qué correo te has registrado. Es la llave de todo lo demás.

2 Crear el proyecto

1. Pulsa **New project**.
2. En *Name* escribe `metalplafer360`.
3. En *Database Password* pulsa **Generate a password** y **cópiala a un sitio seguro**. No la vas a necesitar para el día a día, pero si la pierdes no se puede recuperar.
4. En *Region* elige **West EU (Ireland)** o **Central EU (Frankfurt)**: son las más cercanas y la aplicación irá más rápida.
5. Pulsa **Create new project** y espera un par de minutos.

> **Los proyectos gratuitos se duermen**
>
> Si nadie entra durante una semana, Supabase pausa el proyecto y la aplicación dejará de conectar. Se despierta entrando en supabase.com y pulsando **Restore project**. Usándola a diario no pasa nunca.

3 Ejecutar el SQL

Ahora vas a crear la aplicación por dentro. Son unos archivos de texto que se copian y se pegan en Supabase, **en orden**. No hay que entender lo que dicen.

1. En el menú de la izquierda de Supabase, pulsa **SQL Editor**.
2. Pulsa **New query**.
3. Abre en GitHub el archivo `supabase/sql/01_schema.sql`, pulsa el botón de copiar (el icono de dos hojas, arriba a la derecha) y pégalo en la ventana de Supabase.
4. Pulsa **Run** (abajo a la derecha). Debe aparecer **Success. No rows returned**.
5. Borra la ventana y repite con cada archivo de la lista, en este orden exacto.

ORDEN	ARCHIVO	QUÉ CREA
1	`01_schema.sql`	Las tablas base
2	`02_functions.sql`	Alta de usuarios, numeración anual e historial
3	`03_security.sql`	Los permisos
4	`04_storage.sql`	El almacén de archivos
5	`06_schema_clientes_fichas.sql`	Clientes, contactos y fichas
6	`07_funciones_clientes_fichas.sql`	Códigos, estados y sugerencias
7	`08_seguridad_clientes_fichas.sql`	Permisos de clientes y fichas
8	`09_schema_proyectos.sql`	Proyectos y subestados
9	`10_funciones_proyectos.sql`	Fases y conversión de presupuestos
10	`11_seguridad_proyectos.sql`	Permisos de proyectos
11	`12_schema_ordenes.sql`	Órdenes de trabajo y partes
12	`13_funciones_ordenes.sql`	Revisión, devolución y avisos
13	`14_seguridad_ordenes.sql`	Permisos de las órdenes
14	`15_schema_material.sql`	Material, avisos del taller y cobros
15	`16_funciones_material.sql`	Proveedores, retrasos y calendario
16	`17_seguridad_material.sql`	Permisos de material y facturación
17	`18_schema_informes.sql`	Registro de copias y almacenamiento
18	`19_funciones_informes.sql`	Panel, informes, copias y restauración
19	`20_seguridad_informes.sql`	Permisos de informes y copias
20	`22_correcciones_auditoria.sql`	Correcciones de seguridad e índices
21	`24_google_oauth.sql`	La conexión con Google Drive (apartado 10)
22	`25_correccion_limpiar_demo.sql`	Corrección de «Limpiar datos demo»
23	`26_schema_partes.sql`	Trabajadores asignados a proyectos y partes de trabajo
24	`27_funciones_partes.sql`	Guardar, enviar, validar y rechazar partes
25	`28_seguridad_partes.sql`	Permisos de los partes

26	`29_documentacion_borrado.sql`	Las dos categorías de documentación y el borrado real
27	`30_calendario_planificacion.sql`	Calendario y planificación automática
28	`31_obras.sql`	Las obras: agrupan proyectos, sus cuentas y su facturación
29	`32_trabajador_avisos_material.sql`	Avisos de taller facturables, pedir material y el calendario del trabajador
30	`33_revision_permisos.sql`	Revisión de permisos: datos personales, almacén y usuarios desactivados

> **Los archivos 05, 21 y 23 no se ejecutan ahora.** El `05` son consultas de apoyo para más adelante; el `21` programa la copia automática (apartado 11) y el `23` pone los datos de ejemplo (apartado 17).

> Si sale un aviso amarillo que empieza por `NOTICE:`, no pasa nada: es informativo. Solo es un problema si aparece en rojo la palabra **ERROR**.

Comprobar que ha ido bien

Pega esto en una ventana nueva y pulsa Run. Las cuatro filas deben decir **CORRECTO**:

```
select 'Tablas creadas' as comprobacion,
       case when count(*) = 18 then 'CORRECTO' else 'FALTAN TABLAS' end as resultado
  from pg_tables where schemaname = 'public'
   and tablename in ('profiles','app_settings','code_counters','audit_log',
                     'clients','client_contacts','fichas','documents','comments',
                     'projects','project_substatuses','work_orders',
                     'work_order_workers','notifications','materials',
                     'material_requests','payments','backups')
union all
select 'Seguridad activada',
       case when bool_and(rowsecurity) then 'CORRECTO' else 'HAY TABLAS SIN PROTEGER' end
  from pg_tables where schemaname = 'public'
union all
select 'Almacén de archivos',
       case when exists (select 1 from storage.buckets
                         where id = 'documentos' and not public)
            then 'CORRECTO' else 'FALTA' end
union all
select 'Panel, informes y copias',
       case when count(*) = 11 then 'CORRECTO' else 'FALTAN FUNCIONES' end
  from pg_proc p join pg_namespace n on n.oid = p.pronamespace
 where n.nspname = 'public'
   and p.proname in ('dashboard_summary','report_projects','report_projects_by_phase',
                     'report_hours_by_worker','report_hours_by_project','report_materials',
                     'report_suppliers','start_backup','finish_backup','restore_data',
                     'restore_table');
```

4 Las tablas: qué se guarda y dónde

No hace falta que toques nada de esto, pero conviene saber qué hay dentro por si algún día quieres mirar un dato directamente en Supabase (**Table Editor** en el menú de la izquierda).

TABLA	QUÉ GUARDA
`profiles`	Quién es cada persona y si es administración o trabajador
`clients` · `client_contacts`	Clientes y sus personas de contacto
`fichas`	Presupuestos, visitas y avisos, con su código anual
`projects` · `project_substatuses`	Proyectos, su fase y sus subestados
`work_orders`	Órdenes de fabricación y de montaje
`work_order_workers`	El parte de cada persona: qué hizo y cuántas horas
`materials` · `material_requests`	Material pendiente y lo que comunica el taller
`payments`	Cobros parciales (sin número de factura, a propósito)
`documents`	Fotos, planos, vídeos y firmas, con su papelera
`comments`	Comentarios de texto de fichas, proyectos, órdenes, partes y obras
`project_workers`	Quién está asignado a cada proyecto
`work_reports`	Los partes de trabajo: qué se hizo y cuántas horas
`non_working_days` · `calendar_notes`	Festivos del taller y eventos sueltos del calendario
`works` · `work_extras`	Las obras, que agrupan proyectos, y sus costes extras
`worker_projects` · `worker_fichas`	No son tablas: son las vistas por las que entra el trabajador, escritas sin ninguna columna de dinero
`audit_log`	El historial: quién hizo qué y cuándo. No se puede modificar
`backups`	Qué copia de seguridad se hizo y dónde quedó
`app_settings` · `code_counters`	Datos de la empresa y numeración anual

Aquí no se borra nada

Los clientes, fichas, proyectos, órdenes y materiales se **archivan**: desaparecen del listado pero conservan su historial. Los documentos van a una **papelera** de la que se pueden recuperar. Un cobro apuntado por error se **anula** y se queda tachado a la vista. Y a las personas no se les borra: se les **desactiva**.

Eliminar de verdad existe, pero solo para lo que no arrastra nada: administración puede borrar una ficha, un proyecto, una obra o a un trabajador **si no tienen horas apuntadas, cobros ni facturación**. Si los tienen, la base de datos se niega y explica por qué. Al eliminar una obra, sus proyectos **no** se borran: se quedan sueltos.

5 Los permisos: por qué esto es seguro

Esta es la parte más importante del manual, aunque no tengas que hacer nada.

La seguridad de METALPLAFER360 **no está en la pantalla**. No se trata de esconderle botones al trabajador: eso sería fácil de saltarse. Está en la propia base de datos, con una tecnología que se llama *seguridad a nivel de fila* (RLS, por sus siglas en inglés).

Cada vez que la aplicación pide un dato, PostgreSQL comprueba quién lo pide y **solo le devuelve lo que le corresponde**. Da igual que alguien manipule la aplicación desde el navegador, que escriba una dirección a mano o que use herramientas de programador: el candado está en el servidor, no en el botón.

QUIÉN	QUÉ VE
Administración	Todo
Trabajador	Solo sus órdenes, los proyectos y clientes de esas órdenes, sus propias horas, sus avisos y su historial
Trabajador	**Nunca**: informes, configuración, facturación, cobros, copias de seguridad, numeración ni los datos de otros compañeros

> Esto está comprobado con más de 500 pruebas automáticas que se ejecutan contra una base de datos de verdad, incluidas las que intentan saltarse los permisos a mala fe.

6 El acceso de las personas

En METALPLAFER360 cada persona entra con un **nombre de usuario** y una contraseña. No hace falta que nadie tenga correo electrónico de empresa.

Por dentro, Supabase necesita una dirección de correo para identificar a cada persona, así que la aplicación le añade sola un dominio interno: si el usuario es `juan`, por dentro se guarda como `juan@metalplafer.com`. **No se envía ningún correo electrónico nunca**: es solo un identificador.

Desactivar los registros públicos

Para que nadie pueda darse de alta por su cuenta:

1. En Supabase, menú de la izquierda → **Authentication**.
2. Pestaña **Sign In / Providers** (o **Providers**).
3. Busca **Allow new users to sign up** y **desactívalo**.
4. Guarda.

> Hazlo **después** de crear el primer administrador (apartado 8), no antes, o no podrás crearlo.

7 El almacén de archivos

Las fotografías de obra, los planos, los vídeos y las firmas no se guardan en la base de datos: van a un almacén aparte que el archivo `04_storage.sql` ya creó, y que se llama `documentos`.

Ese almacén es **privado**. Nadie puede abrir una fotografía con solo conocer su dirección: la aplicación pide un enlace temporal cada vez, y ese enlace caduca. Y un trabajador solo obtiene enlaces de los archivos de sus propias órdenes.

El tamaño máximo por archivo

De serie son **100 MB**, que da de sobra para fotos y vídeos de móvil. Se cambia desde la propia aplicación, en **Configuración → Almacenamiento**, y el límite lo aplica la base de datos, no solo la pantalla.

En **Configuración → Almacenamiento** también ves cuánto ocupan las fotografías, los vídeos, las firmas y los documentos, y cuánto hay esperando en la papelera.

8 Crear el primer administrador

La **primera** persona que se da de alta se convierte automáticamente en administración. Las siguientes nacen como trabajadores. Esto es a propósito: así nadie puede concederse permisos por su cuenta.

1. En Supabase: **Authentication → Users → Add user → Create new user**.
2. En *Email* escribe `salvi@metalplafer.com` (el nombre de usuario que quieras, seguido de `@metalplafer.com`).
3. En *Password* pon una contraseña de al menos 8 caracteres y apúntala.
4. Marca **Auto Confirm User**.
5. Pulsa **Create user**.

Para comprobar que ha nacido como administración, ejecuta en el SQL Editor:

```
select full_name as nombre, username as usuario, role as rol, active as activo
  from public.profiles;
```

Debe salir una fila con `rol = admin`.

> Si por lo que sea saliera `worker`, se arregla con:
> `update public.profiles set role = 'admin' where username = 'salvi';`

9 Las claves de conexión

La aplicación necesita saber dónde está su base de datos. Son dos datos que se copian de Supabase a GitHub.

Cogerlos en Supabase

1. **Project Settings** (la rueda dentada, abajo a la izquierda).
2. Apartado **Data API**: copia **Project URL**. Es algo como `https://abcdefgh.supabase.co`.
3. Apartado **API Keys**: copia la clave **anon** (también llamada *public*). Es un texto larguísimo.

> **La clave «anon» sí puede ir a GitHub**
>
> Parece una contraseña, pero no lo es: solo sirve para decir «soy la aplicación de Metalplafer». Quien la tenga no puede ver ni un dato, porque los permisos los aplica la base de datos según quién haya iniciado sesión. Está diseñada para viajar al navegador.

> **La clave «service_role» NO va a GitHub. Nunca.**
>
> En esa misma pantalla verás otra clave llamada `service_role` o *secret*. Esa **se salta todos los permisos**. No se pega en GitHub, ni en el programa, ni en un correo, ni en un WhatsApp. Solo se usa dentro de Supabase, y de eso se encarga el apartado 10.

Pegarlos en GitHub

1. Entra en el repositorio en GitHub.
2. **Settings** → **Secrets and variables** → **Actions** → pestaña **Variables**.
3. Pulsa **New repository variable** y crea estas tres:

NOMBRE	VALOR
`VITE_SUPABASE_URL`	La Project URL que copiaste
`VITE_SUPABASE_ANON_KEY`	La clave anon
`VITE_LOGIN_DOMAIN`	`metalplafer.com`

> Fíjate en que sea la pestaña **Variables** y no **Secrets**. Si las pones en Secrets, la aplicación se publicará sin conexión y en la pantalla de acceso dirá que no está configurada.

10 Google Drive (opcional)

Esto se puede dejar para otro día. Sin hacer nada de este apartado, la aplicación ya hace copias: en **Configuración → Copias de seguridad** tienes el botón **GENERAR BACKUP AHORA**, que descarga un ZIP a tu ordenador.

Lo que se configura aquí es que esa copia se haga **sola, todas las noches, y se guarde en Google Drive**, fuera de Supabase.

> **Por qué hay que autorizar la cuenta**
>
> Google no da espacio de almacenamiento propio a las «cuentas de servicio», así que una de ellas no puede guardar nada en «Mi unidad» de una cuenta Gmail normal. Por eso METALPLAFER360 pide autorización a la cuenta de la empresa **una sola vez**, y a partir de ahí sube las copias con ese permiso, también las de la madrugada.

> Vas a manejar un **secreto de cliente** de Google. No se pega nunca en el repositorio ni en ningún archivo del proyecto: solo en la pantalla de secretos de Supabase. El permiso que Google devuelve al autorizar no lo verás nunca: se guarda cifrado en la bóveda de Supabase sin pasar por el navegador.

10.1 · Crear el cliente OAuth en Google

1. Entra en **console.cloud.google.com** con la cuenta donde quieres las copias (`metalplafer@gmail.com`).
2. **New project** → nómbralo `metalplafer360` → **Create**.
3. En el buscador, **Google Drive API** → **Enable**.
4. **APIs y servicios → Pantalla de consentimiento de OAuth**: tipo **External**, nombre `METALPLAFER360`, y tu correo en los dos campos de contacto.
5. En **Scopes**, añade *solo* este: `https://www.googleapis.com/auth/drive.file`. Es el permiso más pequeño que sirve: la aplicación solo puede tocar sus propios archivos, nada más de tu Drive.
6. **Pulsa PUBLISH APP** hasta que el estado diga **In production**.
7. **Credenciales → Create credentials → OAuth client ID**, tipo **Web application**. En *Authorized redirect URIs* pega exactamente, sin barra al final: `https://TU-REFERENCIA.supabase.co/functions/v1/google-oauth/callback`
8. Google te enseña el **Client ID** y el **Client secret**.

> **El paso 6 no es opcional**
>
> Mientras la aplicación esté en «Testing», Google caduca el permiso **cada 7 días** y la copia automática dejaría de funcionar al cabo de una semana. Como solo se usa `drive.file`, que no es un permiso sensible, publicarla es inmediato y **no hace falta ninguna verificación de Google**.

10.2 · Guardar los secretos en Supabase

Edge Functions → Secrets, y añade:

NOMBRE	VALOR
`GOOGLE_OAUTH_CLIENT_ID`	El Client ID
`GOOGLE_OAUTH_CLIENT_SECRET`	El Client secret
`BACKUP_CRON_SECRET`	Una frase larga que te inventes

10.3 · Publicar los programas del servidor

```
npm install -g supabase
supabase login
supabase link --project-ref TU-REFERENCIA-DE-PROYECTO
supabase functions deploy google-oauth
supabase functions deploy backup-drive
supabase functions deploy admin-users
```

El proyecto trae un `supabase/config.toml` que permite que la vuelta de Google y la tarea de las 02:00 lleguen sin sesión iniciada. Hace falta, y no deja nada abierto: cada programa comprueba por su cuenta quién llama.

10.4 · Conectar

1. En la aplicación: **Configuración → Copias de seguridad**.
2. Pulsa **Conectar Google Drive** y acepta en la ventana de Google.
3. Si avisa de que la aplicación no está verificada, pulsa **Configuración avanzada → Ir a METALPLAFER360**: es tu propia aplicación.
4. El recuadro debe pasar a **Conectado**, con la cuenta y la carpeta.

> La aplicación **crea ella misma** la carpeta «METALPLAFER360 · Copias de seguridad» en tu Mi unidad. Es necesario: con el permiso `drive.file` solo puede escribir en carpetas creadas por ella. Puedes moverla o renombrarla después; la seguirá encontrando.

11 Las copias de seguridad

A mano, cuando quieras

Configuración → Copias de seguridad → GENERAR BACKUP AHORA. Se descarga un ZIP a tu ordenador con todo lo que hay dentro de la aplicación.

Dentro del ZIP encontrarás:

- Un archivo **CSV por cada tabla**, que se abre con Excel.
- Un `metadatos.json` con lo necesario para restaurar.
- Un índice con la **referencia** de cada foto, vídeo y documento.

> **Las fotografías y los vídeos no se convierten en CSV.** Siguen guardados en el almacén de Supabase; en el ZIP va su ruta y sus datos. Es a propósito: meterlos dentro haría la copia enorme y lentísima.

Automática, todas las noches

Solo si has hecho el apartado 10 y Google Drive aparece como «Conectado». En el SQL Editor de Supabase, una vez:

```
select vault.create_secret('https://TU-REFERENCIA.supabase.co', 'm360_project_url');
select vault.create_secret('la-misma-frase-de-BACKUP_CRON_SECRET', 'm360_backup_secret');
```

Y después ejecuta el archivo `supabase/sql/21_backup_automatico.sql`. Debe aparecer un aviso que dice *«Copia automática programada: todos los días a las 02:00 (hora de Madrid)»*.

Se guardan **30 días** de copias, y se puede cambiar en **Configuración → Copias de seguridad**. Las automáticas más antiguas se borran solas de Drive; las que te descargues tú no se tocan.

Comprobar que se sigue haciendo

De vez en cuando, ejecuta en el SQL Editor:

```
select case when max(started_at) is null
            then 'NUNCA se ha hecho una copia automática'
            when max(started_at) > now() - interval '36 hours'
            then 'CORRECTO: al día'
            else 'ATENCIÓN: la última fue el '
                 || to_char(max(started_at) at time zone 'Europe/Madrid', 'DD/MM/YYYY')
       end as copia_automatica
  from public.backups
 where kind = 'automatico' and status = 'completado';
```

12 Restaurar una copia

Solo hace falta si se ha perdido o estropeado información.

> ⚠️ **Esta operación puede modificar información actual**
>
> Los datos de la copia se vuelcan sobre los de ahora: lo que coincide se actualiza y lo que falta se añade. **No se borra nada**, pero lo que se sobrescriba no vuelve. Genera una copia antes, por si acaso.

1. **Configuración → Copias de seguridad**.
2. Antes de nada, pulsa **GENERAR BACKUP AHORA** para tener el estado actual.
3. Baja hasta **Restaurar una copia** y pulsa **Elegir archivo de copia**.
4. La aplicación te dice de cuándo es la copia y cuántos registros trae.
5. Marca **qué apartados** quieres restaurar: puedes traerte solo los clientes, solo los proyectos, o todo.
6. Pulsa **Restaurar lo marcado** y confirma.

> Restaurar **no crea usuarios nuevos**. Si en la copia aparece una persona que ya no existe, sus datos no se recrean: hay que darla de alta otra vez (apartado 16). Es a propósito, porque el acceso vive en Supabase, no en la copia.

Toda restauración queda registrada en el historial, con quién la hizo y qué apartados tocó, y la numeración de códigos se reajusta sola para que el siguiente presupuesto no repita un número.

13 GitHub

GitHub es donde vive el programa (no los datos: los datos están en Supabase). Desde ahí se publica solo, sin que tengas que hacer nada más.

CARPETA	QUÉ CONTIENE
`src/`	Las pantallas de la aplicación
`supabase/sql/`	Todo el SQL que se ejecuta en Supabase
`supabase/functions/`	Los dos programas del servidor (ahí viven las credenciales)
`public/brand/`	Donde van los logos oficiales de Metalplafer
`docs/`	Este manual y la guía de puesta en marcha
`tests/`	Las pruebas automáticas

Los logos

Copia los archivos oficiales en `public/brand/` con estos nombres exactos: `logo-azul.png` (para fondos claros) y `logo-blanco.png` (para fondos oscuros). Aparecerán solos en la pantalla de acceso, el menú lateral y la cabecera del móvil. El logo no se redibuja ni se modifica nunca.

14 Publicar la aplicación

1. En GitHub: **Settings** → **Pages**.
2. En *Source* elige **GitHub Actions**.
3. Ve a la pestaña **Actions** → **Publicar METALPLAFER360** → **Run workflow**.
4. Espera dos o tres minutos a que salga la marca verde.

La dirección de la aplicación será algo como `https://TU-USUARIO.github.io/metalplafer360/`. Esa es la que se reparte a la gente del taller.

> A partir de ahora, cada vez que se suba un cambio a GitHub la aplicación se reconstruye y se publica sola. No hay que hacer nada más.

15 Entrar por primera vez

1. Abre la dirección de la aplicación.
2. Escribe el usuario (`salvi`, sin el `@metalplafer.com`) y la contraseña.
3. Pulsa **Entrar**.

Los primeros diez minutos

1. Ve a **Configuración → Empresa** y rellena el nombre, el CIF, el teléfono y la dirección. Eso es lo que sale en los informes en PDF.
2. En **Configuración → Aplicación**, pulsa **Poner datos de ejemplo** y date una vuelta por la aplicación con datos dentro (apartado 17).
3. Cuando te hayas hecho a ella, pulsa **Limpiar datos demo**.
4. Da de alta a la gente del taller (apartado 16).
5. Instala la aplicación en el móvil: en Android, menú del navegador → «Añadir a pantalla de inicio»; en iPhone, botón Compartir → «Añadir a pantalla de inicio».

> La aplicación **necesita Internet**, y es a propósito: en una obra es peor trabajar con datos viejos que no poder entrar. Instalada en el móvil solo se guardan las pantallas y los iconos, nunca los datos de la empresa.

16 Dar de alta trabajadores

Desde la aplicación (si hiciste el apartado 10.4)

1. **Trabajadores → Nuevo trabajador**.
2. Escribe el nombre, el usuario (`juan`) y una contraseña.
3. La persona ya puede entrar. Nace como **trabajador**; para darle permisos de administración, usa el selector de su fila.

A mano, desde Supabase

1. **Authentication → Users → Add user**.
2. Email: `juan@metalplafer.com`. Contraseña: la que sea.
3. Marca **Auto Confirm User** y crea.
4. Entra en la aplicación, ve a **Trabajadores** y corrige su nombre completo.

> **A nadie se le borra**
>
> Cuando alguien deja la empresa, se le **desactiva** desde **Trabajadores**. Deja de poder entrar, pero sus horas y su historial siguen ahí. Si intentas borrarlo desde Supabase, la base de datos se negará: es una protección puesta a propósito para que no se pierdan horas ya trabajadas y facturadas.

17 Los datos de ejemplo

Para aprender a usar la aplicación, enseñarla o hacer pruebas sin miedo, hay un taller inventado listo para poner.

1. Ejecuta una vez en el SQL Editor el archivo `supabase/sql/23_datos_demo.sql`.
2. En la aplicación: **Configuración → Aplicación → Poner datos de ejemplo**.

Se crean 10 clientes, 15 fichas, 8 proyectos, 6 trabajadores, 20 órdenes de trabajo, material, documentos y cobros, con fechas recientes.

> **Limpiarlo es completamente seguro**
>
> El botón **Limpiar datos demo** borra **exactamente** lo que se creó como ejemplo y nada más. Cada fila lleva una marca por dentro, y la limpieza solo mira esa marca. Lo que haya hecho la empresa de verdad no se toca, aunque esté mezclado.

Los trabajadores de ejemplo se llaman `demo.jordi`, `demo.marc`… y nacen sin contraseña; si quieres entrar como uno para ver el móvil, pónsela desde Trabajadores. Los documentos de ejemplo son solo la ficha del archivo: aparecen en los listados, pero no hay nada que descargar.

18 Actualizaciones

Cuando llegue una versión nueva del programa:

1. Sube los archivos nuevos a GitHub (o acepta el cambio que te propongan).
2. La aplicación se reconstruye y se publica sola en un par de minutos.
3. Si la versión trae archivos SQL nuevos, ejecútalos en Supabase **en orden**, como en el apartado 3. Los que ya ejecutaste no hay que repetirlos.
4. En el móvil, la aplicación avisa sola de que hay una versión nueva y se actualiza al recargar.

> Los archivos SQL están pensados para poder ejecutarse más de una vez sin romper nada. Si dudas si ya ejecutaste uno, ejecutarlo otra vez es seguro.

19 Si algo no funciona

QUÉ VES	QUÉ HACER
«La aplicación todavía no está configurada»	Las variables de GitHub están vacías o mal escritas. Revisa el apartado 9: deben estar en **Variables**, no en Secrets. Después, Actions → Run workflow otra vez.
«Usuario o contraseña incorrectos»	Revisa en Authentication → Users. Puedes cambiar la contraseña con ⋯ → Reset password.
«Tu usuario no tiene perfil»	Se creó antes de ejecutar el SQL. Bórralo en Authentication → Users y créalo otra vez.
«Tu usuario está desactivado»	Reactívalo desde Trabajadores, o con `update public.profiles set active = true where username = 'juan';`
«No se ha podido conectar con el servidor»	Sin Internet, o el proyecto de Supabase está pausado: entra en supabase.com y pulsa **Restore project**.
El trabajador no ve ninguna orden	No se le ha asignado ninguna, o son de días futuros. Las futuras se ven, pero no se rellenan hasta su fecha.
«Falta configurar la conexión con Google»	No están puestos los secretos de Google. Repasa el apartado 10.2. El botón GENERAR BACKUP AHORA funciona igual sin eso.
«Google ya no acepta el permiso guardado»	Se retiró el acceso desde la cuenta de Google, o la aplicación se quedó en «Testing» y Google caduca el permiso cada 7 días. Publícala (paso 6 del apartado 10.1) y vuelve a pulsar **Conectar Google Drive**.
«Google no ha dado acceso»	Un fallo momentáneo de Google. Vuelve a intentarlo más tarde; la copia automática lo reintenta al día siguiente por su cuenta.
«No tienes permisos» al limpiar los datos de ejemplo	Falta ejecutar `supabase/sql/25_correccion_limpiar_demo.sql`. Ejecútalo una vez en el SQL Editor y vuelve a pulsar el botón. No borra ni cambia ningún dato.
La copia automática no se hace	No se ejecutó el archivo 21, o faltan los secretos de la bóveda. Repasa el apartado 11.
El Excel se ve todo en una columna	Excel está en inglés. Ábrelo con Datos → Desde texto/CSV y elige el punto y coma como separador. Los informes en `.xlsx` no tienen este problema.
No se puede borrar algo	Es correcto: aquí no se borra nada. Los clientes, proyectos, órdenes y materiales se **archivan**; los documentos van a la **papelera**; los cobros se

	anulan; a las personas se les **desactiva**.
Un código de presupuesto o proyecto está mal	Los códigos no se pueden cambiar a propósito: identifican el expediente en presupuestos y facturas de papel. Si hay que corregir la numeración de las próximas fichas, se hace en Configuración → Numeración.

Consultas de apoyo

El archivo `supabase/sql/05_verificacion.sql` tiene casi treinta consultas preparadas y explicadas: ver quién puede entrar, cambiar un rol, mirar el historial, las órdenes abiertas, el material retrasado, la facturación pendiente o el estado de las copias de seguridad. Se copian y se pegan en el SQL Editor.

> **Y si te pierdes**
>
> Toda la aplicación deja rastro: en el historial de cada ficha, proyecto y orden aparece quién hizo qué y cuándo. Nada se pierde sin que quede constancia.